Week1_예습과제_명재서

파머완 1장 : 파이썬 기반의 머신러닝과 생태계 이해

01 머신러닝의 개념

- **머신러닝** : 애플리케이션을 수정하지 않고도 데이터를 기반으로 패턴을 학습하고 결과를 예측하는 알고리즘 기법을 통칭함.
- 개발자가 데이터나 업무 로직의 특성을 직접 감안한 프로그램을 만들 경우, 난이도와 개발 복잡도가, 너무 높아질 수 밖에 없는 분야에서 급속하게 발전 중.

지도학습

→ 분류, 회귀, 추천 시스템, 시각/음성 감지/인지, 텍스트 분석, NLP

비지도학습

→ 클러스터링, 차원 축소, 강화학습

머신러닝에서 중요한 요소 = 데이터 + 머신러닝 알고리즘

머신러닝의 가장 큰 단점 : 데이터에 매우 의존적.
▼ 머신러닝을 개선할 수 있는 방안

1. 최적의 머신러닝 알고리즘
2. 최적의 모델 파라미터 구축
3. 데이터를 이해하고 효율적으로 가공, 처리, 추출해 최적의 데이터를 기반으로 알고리즘을 구동할 수 있도록 준비하는 능력

파이썬과 R기반의 머신러닝 비교

R = 통계 전용 프로그램 언어

오랜 기간 동안 많은 R 사용자들이 생성하고 검증해 온 다양한 많은 통계 패키지를 보유한다는 것이 장점.

파이썬 = 다양한 영역에서 사용되는 개발 전문 프로그램 언어.

직관적인 문법 / 객체지향과 함수형 프로그래밍 모두를 포괄하는 유연한 프로그램 아키텍쳐, 다양한 라이브러리 등의 큰 강점을 가짐.

02 파이썬 머신러닝 생태계를 구성하는 주요 패키지

▼ 사이킷런

데이터 마이닝 기반의 머신러닝 패키지

▼ 넘파이

: 행렬과 선형대수를 다루는 패키지

- 행렬 기반의 데이터 처리에 특화, 일반적 데이터 처리에는 부족.

▼ 사이파이

: 자연과학과 통계를 위한 다양한 패키지

▼ 판다스

: 파이썬 세계의 대표적인 데이터 처리 패키지

- 2차원 데이터 처리에 특화, 쉽게 시각화 가능

▼ 맷플롭립

: 파이썬의 대표적인 시각화 패키지

- 서드파티 라이브러리 = 외부 개발자가 만든 라이브러리

주피터 노트북 (아이파이썬툴) = 프로그래밍 + 설명적 요소

→ 중요 코드 단위로 설명 / 코드 수행 / 결과 제시 = 직관적으로 어떤 코드가 어떤 역할을 쉽게 이해할 수 있음.

03 넘파이

: 파이썬에서 선형대수 기반의 프로그램을 쉽게 만들 수 있도록 지원하는 대표적인 패키지

- 루프를 사용하지 않고 대량 데이터의 배열 연산을 가능하게 하므로 빠른 배열 연산 속도를 보장함.

➡️ 빠른 계산 능력이 중요한 대량 데이터 기반의 과학과 공학 패키지는 넘파이에 의존함.

- C/C++ 과 같은 저수준 언어 기반의 호환 API를 제공함.
- 배열 기반의 연산 & 다양한 데이터 핸들링 기능을 제공.
- 편의성 / 다양한 API 지원 측면에서 파이썬의 대표적 데이터 처리 패키지인 판다스의 편리성에 미치지 못함.

✅하지만, 많은 머신러닝 알고리즘이 넘파이를 기반으로 작성되어 있고, 이들 알고리즘의 입출력 데이터를 넘파이 배열 타입으로 사용하기 때문에 넘파이를 이해하는 것은 파이썬 기반의 머신러닝에서 매우 중요함!!

▼ 넘파이 ndarray 개요

- 다차원 배열을 쉽게 생성하고 다양한 연산 수행 가능.
- array 함수는 파이썬의 리스트와 같은 다양한 인자를 입력 받아서 ndarray로 변환하는 기능을 수행함.
- 생성된 shape 변수는 ndarray의 크기 (행과 열의 수)를 튜플 형태로 가지고 있어, 배열의 차원까지 알 수 있음.
- np.array의 사용법 : ndarray로 변환을 원하는 객체를 인자로 입력하면 ndarray를 반환.
- ndarray.shape : ndarray의 차원과 크기를 튜플 형태로 나타내줌.

- 머신러닝 알고리즘과 데이터 세트 간의 입출력과 변환을 수행하다 보면, 데이터의 차원을 명확히 구분해서 요구하는 경우가 있으므로, 로우 &칼럼으로 이뤄진 데이터의 차원을 구분하는 것이 중요함. (데이터 값으로는 동일하나, 차원이 달라서 오류가 발생하는 경우!!)

▼ ndarray의 데이터 타입

- ndarray의 데이터 값은 숫자, 문자열, 불 값이 모두 가능함. 하지만, ndarray 내의 데이터 타입은 그 연산의 특성상 같은 데이터 타입끼리만 가능하다.
- ndarray 내의 데이터 타입은 dtype 속성으로 확인 가능.
- 만약 다른 데이터 유형이 섞여있는 리스트를 ndarray로 변경하면 데이터 타입이 더 큰 데이터 타입으로 변환해 연산을 수행함.
- astype( )에 인자로 원하는 타입을 문자열로 지정하면, 데이터 값의 타입 변경 가능. - 대용량 데이터의 ndarray를 만들며 메모리를 절약할 때 보통 이용됨. (수행 속도가 느려지거나 메모리 부족으로 인한 오류 방지)

arange()

: 0부터 함수 인자 값 -1까지의 값을 순차적으로 ndarray의 값으로 변환해 줌.

zeros()

: 함수 인자로 튜플 형태의 shape 값을 입력하면 0으로 채운 ndarray를 반환해 줌.

ones()

: 함수 인자로 튜플 형태의 shape 값을 입력하면 1로 채운 ndarray를 반환해 줌.

reshape()

: ndarray의 차원과 크기를 변경해줌. *지정된 사이즈로 변경이 불가능하면 오류를 발생함.

Indexing - 데이터 세트 선택하기

- 인덱스 값을 통한 단일 값 추출.

- 로우와 칼럼 위치의 인덱스를 통한 다차원 ndarray에서 단일 값 추출.
- axis 0 = 로우 방향의 축 / axis 1 = 칼럼 방향의 축
- 슬라이싱 - 연속한 데이터를 추출 (시작 : 종료)
- 펜시 인덱싱 - 인덱스 집합을 지정하면 해당 위치의 인덱스에 해당하는 ndarray를 반환.
- 불린 인덱싱 - 조건 필터링 & 검색을 동시에 할 수 있어 자주 사용됨. (반복문, 조건문보다 간단)

행렬 정렬

1. np.sort( ) : 원 행렬은 유지한 채, 정렬된 행렬을 반환.
2. ndarray.sort( ) : 원 행렬 자체를 정렬한 형태로 반환 & 반환 값 = None

내림차순으로 정렬 [::-1]

3. np.argsort( ) : 원본 행렬의 정렬시 인덱스 값 구하기

선형대수 연산

- 행렬 내적 (곱) : np.dot( )
- 전치 행렬 : transpose( )

---

04 판다스

: 2차원 데이터를 효율적으로 가공/처리할 수 있게 해주는 라이브러리

- csv, tab과 같은 다양한 유형의 분리 문자로 칼럼을 분리한 파일을 손쉽게 DataFrame으로 로딩할 수 있게 해줌.

→ read_csv( ), read_table( ), read_fwf( )

info( )

: 데이터 건수, 데이터 타입, Null 건수 등을 알 수 있음.

describe( )

: 칼럼별 숫자형 데이터 값의 n-percentile 분포도, 평균값, 최댓값, 최솟값을 나타냄.

- 오직숫자형 칼럼의 분포도만 조사하며 자동으로 object 타입의 칼럼은 출력에서 제외시킴.
- 개략적인 수준의 분포도를 확인할 수 있어 유용.

value_counts( )

: 해당 칼럼값의 유형과 건수를 확인 가능

- 데이터의 분포도를 확인하는 데 매우 유용
- Null 값을 무시하고 결과를 내놓기 쉽다는 점을 유의해야 함.

DataFrame과 리스트, 딕셔너리, 넘파이 ndarray는 상호 변환 가능.

DataFrame의 칼럼 데이터 세트 생성과 수정 [ ] 연산자를 통해 가능.

drop( )

: DataFrame 데이터 삭제

- 로우를 삭제할 때는 axis = 0, 칼럼을 삭제할 때는 axis = 1